1) Find 10 different model providers and their models released in the last 12 months

Details to collect:
Model Provider | Model | Release Date | Cutoff Date | Cost of tokens | Context Window | Type | When to use this model |

10 AI model providers and their latest models (November 2025–November 2026)

The research period is 2 November 2025 to 2 November 2026. I've selected 10 different AI model providers, with one representative model from each provider.

The comparison uses the following conventions:

- Cost of tokens: USD per 1 million input and output tokens.
- Context window: Maximum tokens the model can process in its context.
- Cutoff date: Training-data knowledge cutoff, not the model's release date. Where the provider does not disclose it, I mark it as *Not publicly disclosed*.
- Type: Model category and availability, such as proprietary, open-weight, reasoning, multimodal or coding.
- When to use: Practical recommendations based on the model's intended strengths.

Release trackers updated on 1–2 November 2026 provide a useful cross-provider starting point, with model announcements and pricing records.

Developers Digest
+2

Comparative research table

All prices below are indicative standard API rates, in USD per 1 million tokens. They exclude caching, batch discounts, hosting charges and taxes unless stated otherwise.

#	Model provider	Model	Release date	Cutoff date
1	OpenAI	GPT-5.2	11 Dec 2025	31 Aug 2025
2	Anthropic	Claude Opus 4.5	24 Nov 2025	Aug 2025
3	Google DeepMind	Gemini 3.8 Flash	Sep 2026	Not disclosed
4	xAI / SpaceXSI	Grok 4.7	21 Sep 2026	Not disclosed
5	Alibaba Cloud	Qwen3.8 Max	2 Aug 2026	Not disclosed
6	DeepSeek	DeepSeek-V4-Pro	24 Apr 2026	Not disclosed
7	Mistral AI	Mistral Large 4	6 Oct 2026	Not disclosed
8	Z.ai	GLM-5.3 Prime	23 Sep 2026	Not disclosed
9	Amazon Web Services	Amazon Nova 2 Lite	2 Dec 2025	Oct 2025
10	NVIDIA	Nemotron 3.5 Super VL	15 Oct 2026	Not disclosed

2) Read 2–3 articles about how transformers work?
Goal to understand:
(a) Self-attention mechanism
(b) Vector embedding
(c) Feed-forward neural network

A Transformer does this through three important steps:

1. Vector Embedding: Convert words into numbers.
2. Self-Attention: Understand how words relate to one another.
3. Feed-Forward Neural Network: Process and improve the understanding of each word.

Let's understand each one.

1. Vector Embedding – Converting words into numbers

The problem: Computers don't naturally understand words like humans do. They work with numbers.

So, how can a computer understand the word "cat"?

We convert each word into a list of numbers called a vector embedding.

Imagine this:

Human language

CAT

Computer representation

[0.8, 0.2, 0.9, 0.4]

Illustrative numbers, not actual model values

Every word or token gets its own numerical representation.

For example:

Word	Example embedding
Cat	[0.8, 0.2, 0.9]
Dog	[0.7, 0.3, 0.8]
Car	[0.1, 0.9, 0.2]
Milk	[0.6, 0.4, 0.5]

Notice that cat and dog have somewhat similar representations in this illustration because they are conceptually related.

Why is this useful?

The computer can work with these numbers and learn relationships between words.

Think of embedding as:

Giving every word a numerical identity that captures something about its meaning.

2. Self-Attention – Understanding relationships between words

Now comes the interesting part!

Consider this sentence:

"The cat drank milk because it was thirsty."

What does "it" refer to?

A human understands that "it" refers to the cat.

But how does a computer figure this out?

This is where self-attention helps.

Self-attention allows each word to look at other words in the sentence and determine which ones are important for understanding its meaning.

Imagine self-attention working like this:

Sentence: The cat drank milk because it was thirsty.

The

cat

drank

milk

because

it

was

thirsty

When processing "it", the model looks at other words:

cat

Highly relevant

milk

Less relevant

drank

Less relevant

Result: "It" is understood in relation to "cat".

Simplified illustration: actual attention calculates numerical relevance scores, rather than using fixed rules.

There are three important terms in self-attention:

Term	Simple meaning
Query (Q)	What information am I looking for?
Key (K)	What information does each word offer?
Value (V)	What information should I take from that word?

Think of searching Google:

- Query = Your search question.
- Key = Labels describing available information.
- Value = The actual information you retrieve.

Why is this useful?

It helps the model understand context rather than treating every word independently.

For example, the word "bank" means something different in:

- River bank
- Money bank

Self-attention helps the model understand which meaning is relevant from the surrounding words.

3. Feed-Forward Neural Network – Processing the information

Now the computer has:

- Converted words into numbers (embedding).
- Understood relationships between words (self-attention).

What happens next?

The information goes through a Feed-Forward Neural Network (FFN).

Think of it as a processing unit that takes the information for each token and transforms it into a more useful representation.

Imagine an employee working on information collected from different sources.

- Self-attention = Collecting relevant information from colleagues.
- Feed-forward network = Analysing and processing the collected information.
- Output = Updated understanding.

Input from self-attention

Numerical representation of "cat" with contextual information

Feed-Forward Neural Network

Mathematical transformations using learned weights and an activation function

Output

A transformed representation of "cat"

There is one important difference:

- Self-attention allows tokens to exchange information with other tokens.

- Feed-forward networks process each token's representation independently, using the same network weights.

Why is this useful?

Attention helps gather relevant context. The feed-forward network further processes that context and helps the model learn complex patterns.

4. Putting all three together

Now let's see how they work together.

How a Transformer works

Step 1: Vector Embedding

Convert tokens into numerical vectors.

Step 2: Self-Attention

Understand relationships between tokens.

Step 3: Feed-Forward Network

Process each token's representation.

Output of one Transformer block

Passed to the next Transformer block.

This sequence is repeated across multiple Transformer blocks. Real blocks also include residual connections and normalization.

The three things to remember:

- Embedding = Give words numbers.
- Self-attention = Understand relationships.
- Feed-forward = Process the information.